

1주 · 리눅스 & Docker

리눅스 커맨드 · 셸 스크립트 · Docker · Compose (5차시, 20H)

1-4 · Dockerfile 작성 및 이미지 최적화

1-4 · Dockerfile 작성 및 이미지 최적화

Dockerfile로 파이썬 앱을 이미지화하고, 베이스 이미지 선택과 레이어 순서 조정으로 크기 · 빌드 시간을 줄입니다.

학습 목표

- FROM · WORKDIR · COPY · RUN · CMD 지시어의 역할을 설명할 수 있습니다.
- 레이어 캐시의 동작 원리를 이해하고 빌드 로그에서 CACHED를 찾을 수 있습니다.
- 파이썬 앱을 Dockerfile로 빌드해 태그를 붙여 실행할 수 있습니다.
- 베이스 이미지 교체와 레이어 순서 조정으로 이미지를 최적화할 수 있습니다.

시간 배분 (4H)

구분	Chapter	시간
이론	Dockerfile 문법과 레이어	1H
실습	이미지 빌드 실습	1.5H
실습	이미지 최적화	1H
정리 · 회고	최적화 전후 비교 공유 · 질의응답	0.5H

참고 사전 준비: [Windows · macOS · 리눅스 모두](#) Docker가 실행 중이어야 하며(`docker version`으로 확인), 3차시의 docker build 실습을 위해 인터넷 연결(베이스 이미지

pull)이 필요합니다.

오늘 만드는 이미지는 5차시 Compose 통합 환경의 app 서비스로 그대로 쓰입니다.

오늘 만들 것

1-3에서는 남이 만든 이미지를 가져다 썼습니다.

오늘은 내 코드를 담은 이미지를 직접 만듭니다.

Dockerfile은 그 제작 설명서입니다.

오늘 만드는 data-app 이미지는 1-5에서 Compose 통합 환경의 app 서비스로 그대로 들어갑니다.

그러니 마지막에 이미지가 정상 실행되는지 꼭 확인하고 마치세요.

Dockerfile 문법과 레이어 (1H)

지시어 여섯 개와 레이어 캐시의 원리를 잡습니다. 순서 하나로 빌드 시간이 몇 분에서 몇 초로 바뀝니다.

1-4 · 이론 1H · Dockerfile 문법과 레이어 슬라이드 11장

슬라이드: <https://raw.githubusercontent.com/aebonlee/materials/main/data-eng/decks/w1d4s1.pdf>

Dockerfile은 레시피 카드입니다

"python 재료로 시작해(FROM), 작업대를 /app으로 정하고(WORKDIR), 재료를 올려놓고(COPY), 손질하고(RUN), 완성되면 이렇게 내놓아라(CMD)"를 순서대로 적은 카드입니다.

docker build는 이 카드를 위에서부터 한 줄씩 실행해 이미지를 굽습니다.

지시어	하는 일	예
FROM	어떤 이미지에서 출발할지	FROM python:3.12-slim
WORKDIR	이후 명령의 기준 디렉터리 (없으면 만들)	WORKDIR /app
COPY	내 컴퓨터 파일을 이미지 안으로	COPY requirements.txt .
RUN	이미지를 만드는 중에 실행할 명령	RUN pip install -r requirements.txt
ENV	환경변수 설정	ENV PYTHONUNBUFFERED=1
EXPOSE	이 이미지가 쓰는 포트를 문서화 (열지는 않음)	EXPOSE 8000
CMD	컨테이너가 시작될 때 실행할 기본 명령	CMD ["python", "app.py"]
ENTRYPOINT	항상 실행되는 고정 명령	ENTRYPOINT ["python"]

주의 RUN과 CMD를 헷갈리기 쉽습니다.

RUN은 이미지를 만드는 동안(빌드 시점) 실행되고 결과가 이미지에 굳습니다.

CMD는 이미지를 만들 때는 실행되지 않고, 나중에 컨테이너를 띄울 때 실행됩니다.

레이어 — Dockerfile 한 줄이 한 층

이미지는 층으로 쌓인 샌드위치입니다.

FROM이 맨 아래 빵이고, 그 위에 COPY·RUN이 한 층씩 올라갑니다.

각 층은 "이전 층에서 무엇이 달라졌는가"만 담고 있습니다.

핵심은 이겁니다.

어떤 층이 바뀌면 그 위의 층은 전부 다시 만들어야 합니다.

아래층이 달라졌으니 그 위에 쌓은 것도 더 이상 유효하지 않기 때문입니다.



- 캐시 전략은 한 문장입니다 — 잘 안 바뀌는 것을 위(먼저), 자주 바뀌는 것을 아래(나중)에 둡니다.
- 의존성 목록(requirements.txt)은 몇 주에 한 번 바뀌지만, 소스 코드는 하루에도 몇 번씩 바뀝니다.
- 그래서 COPY requirements.txt . → RUN pip install → COPY . . 순서가 정답입니다.
- COPY . . 를 먼저 쓰면 코드 한 글자만 고쳐도 pip install이 처음부터 다시 됩니다.

TIP 빌드 로그에서 CACHED 라는 표시를 찾아보세요.

그 줄은 다시 실행하지 않고 지난번 결과를 그대로 쓴 것입니다.

오늘 실습에서 직접 확인합니다.

베이스 이미지 고르기

베이스	크기 (대략)	성격
python:3.12	약 1GB	컴파일러·개발 도구가 다 들어 있음. 빌드는 편하지만 무겁다
python:3.12-slim	약 130MB	실행에 필요한 최소한. 이번 과정의 기본값
python:3.12-alpine	약 50MB	가장 작지만 C 라이브러리가 달라 일부 패키지 설치가 까다롭다

주의 alpine이 작다고 무조건 좋은 것은 아닙니다.

pandas·numpy처럼 C 확장을 쓰는 패키지는 alpine에서 소스 컴파일로 넘어가 빌드가 몇

배 느려지거나 실패합니다.
데이터 쪽은 slim을 기본으로 쓰세요.

빌드 컨텍스트 — 마지막 점(.)의 정체

bash

```
1 docker build -t data-app:0.1 .
2 # ^
3 # 이 점이 "빌드 컨텍스트" 입니다
```

- 빌드 컨텍스트란 도커가 이미지를 만들 때 참고할 수 있는 폴더입니다. COPY는 이 안의 파일만 가져올 수 있습니다.
- 컨텍스트 폴더의 내용이 통째로 도커 데몬에 전송됩니다. 그래서 큰 파일이 섞여 있으면 빌드가 느려집니다.
- .dockerignore 파일에 제외할 것을 적어 두면 전송에서 빠집니다. .git · __pycache__ · 데이터 파일 등이 대상입니다.
- 컨텍스트 밖의 파일은 COPY 할 수 없습니다. ../다른폴더/파일 같은 경로가 안 되는 이유입니다.

이 1시간에서 꼭 가져갈 것

- Dockerfile은 위에서부터 한 줄씩 실행되고, 한 줄이 한 층이 된다.
- 바뀐 층부터 아래(뒤)가 전부 다시 만들어진다.
- 그래서 의존성 설치를 소스 복사보다 먼저 쓴다.
- 데이터 작업의 기본 베이스는 slim이다.
- COPY는 빌드 컨텍스트(마지막 점) 안의 파일만 가져올 수 있다.

이미지 빌드 실습 (1.5H)

파이썬 앱을 이미지로 굽고, 일부러 순서를 잘못 써 보며 캐시가 깨지는 것을 눈으로 확인합니다.

1-4 · 실습 1.5H · 이미지 빌드 실습 슬라이드 11장

슬라이드: <https://raw.githubusercontent.com/aebonlee/materials/main/data-eng/decks/w1d4s2.pdf>

1 이미지에 담을 파이썬 앱을 준비합니다

bash

```
1 mkdir -p ~/data-eng/week1/app
2 cd ~/data-eng/week1/app
3
4 cat > app.py << 'EOF'
5 import platform
6
7 print("데이터 파이프라인 앱이 실행되었습니다.")
8 print("파이썬 버전:", platform.python_version())
9 EOF
10
11 cat > requirements.txt << 'EOF'
12 requests
13 EOF
14
15 ls
```

확인 포인트: ls에 app.py와 requirements.txt 두 파일이 보이면 준비 완료입니다.

2 Dockerfile을 작성합니다

dockerfile

```
1 # Dockerfile - ~/data-eng/week1/app/Dockerfile로 저장
2 FROM python:3.12-slim
3
4 # 컨테이너 안 작업 디렉터리
5 WORKDIR /app
6
7 # 의존성 먼저 설치 (잘 안 바뀌는 층을 먼저)
8 COPY requirements.txt .
9 RUN pip install --no-cache-dir -r requirements.txt
10
11 # 소스 코드는 나중에 복사 (자주 바뀌는 층을 나중에)
12 COPY . .
13
14 # 컨테이너 시작 시 실행할 명령
15 CMD ["python", "app.py"]
```

bash

```
1 cd ~/data-eng/week1/app
2
3 # 위 내용을 파일로 저장 (편집기를 써도 됩니다)
4 cat > Dockerfile << 'EOF'
5 FROM python:3.12-slim
6 WORKDIR /app
7 COPY requirements.txt .
8 RUN pip install --no-cache-dir -r requirements.txt
9 COPY . .
10 CMD ["python", "app.py"]
11 EOF
12
13 cat Dockerfile
```

확인 포인트: cat 결과가 위 여섯 줄과 같으면 됩니다.

--no-cache-dir은 pip이 내려받은 설치 파일을 이미지 안에 남기지 않게 해 크기를 줄이는 옵션입니다.

3 빌드하고 실행합니다

bash

```
1 # 이미지 빌드 (태그: data-app:0.1)
2 docker build -t data-app:0.1 .
3
4 # 만들어진 이미지 확인
5 docker images data-app
6
7 # 실행 (--rm: 종료 시 컨테이너 자동 삭제)
8 docker run --rm data-app:0.1
```

확인 포인트: 빌드가 오류 없이 끝나고, 실행 시 "데이터 파이프라인 앱이 실행되었습니다."와 파이썬 버전 3.12.x가 출력되면 성공입니다.

4 코드를 고치고 재빌드하며 캐시를 관찰합니다

bash

```
1 # app.py에 한 줄 추가
2 echo 'print("이 줄은 재빌드 실험용입니다.")' >> app.py
3
4 # 재빌드 - 빌드 로그에서 CACHED 표시를 찾아보세요
5 docker build -t data-app:0.2 .
6 docker run --rm data-app:0.2
```

확인 포인트: 빌드 로그에서 pip install 단계가 CACHED로 표시되어 순식간에 지나가면 성공입니다.

소스만 바꿨으므로 의존성 층은 캐시가 재사용됩니다.

5 일부러 순서를 잘못 써 봅니다

이론에서 본 "순서가 나쁘면 매번 다시 설치한다"를 직접 겪어 봅니다.
시간을 재서 비교하는 것이 목적입니다.

bash

```

1 # COPY . . 를 먼저 쓰는 나쁜 순서
2 cat > Dockerfile.bad << 'EOF'
3 FROM python:3.12-slim
4 WORKDIR /app
5 COPY . .
6 RUN pip install --no-cache-dir -r requirements.txt
7 CMD ["python", "app.py"]
8 EOF
9
10 # 처음 빌드 (둘 다 오래 걸리는 것이 정상입니다)
11 docker build -t data-app:bad -f Dockerfile.bad .
12
13 # 코드 한 줄 고치고 각각 재빌드해 시간 비교
14 echo 'print("두 번째 실험")' >> app.py
15
16 time docker build -t data-app:good2 -f Dockerfile .
17 time docker build -t data-app:bad2 -f Dockerfile.bad .

```

핵심 포인트: 좋은 순서(Dockerfile)는 pip install이 CACHED로 넘어가 몇 초 만에 끝나고, 나쁜 순서(Dockerfile.bad)는 pip install이 다시 돌아 훨씬 오래 걸립니다.

time이 찍어 준 real 값을 비교해 두세요 — 정리·회고 시간에 공유합니다.

TIP 지금은 패키지가 requests 하나뿐이라 차이가 몇 초 수준입니다.
실무처럼 pandas·pyspark 등 수십 개가 들어가면 이 차이가 몇 분이 됩니다.
하루에 재빌드를 스무 번 한다고 생각해 보세요.

6 .dockerignore로 컨텍스트를 줄입니다

bash

```
1 cat > .dockerignore << 'EOF'
2 # 빌드에 필요 없는 것들 - 전송에서 빠집니다
3 Dockerfile.bad
4 __pycache__/_
5 *.pyc
6 .git/_
7 *.log
8 EOF
9
10 # 다시 빌드하며 첫 줄의 transferring context 크기를 비교해 보세요
11 docker build -t data-app:0.3 .
```

확인 포인트: 빌드 첫 단계의 "transferring context" 크기가 이전보다 줄어들면 성공입니다.

이미지 최적화 (1H)

베이스 이미지 교체와 멀티스테이지 빌드로 크기를 줄이고, 전후를 수치로 비교합니다.

1-4 · 실습 1H · 이미지 최적화 슬라이드 9장

슬라이드: <https://raw.githubusercontent.com/aebonlee/materials/main/data-eng/decks/w1d4s3.pdf>

1 베이스 이미지에 따른 크기 차이를 측정합니다

bash

```
1 cd ~/data-eng/week1/app
2
3 # slim이 아닌 전체 베이스로도 빌드해 비교
4 sed "s/python:3.12-slim/python:3.12/" Dockerfile > Dockerfile.fat
5 docker build -t data-app:fat -f Dockerfile.fat .
6
7 # 크기 비교
8 docker images data-app
```

확인 포인트: docker images 출력에서 data-app:fat이 data-app:0.3보다 수백 MB 이상 큰 것을 확인하면 성공입니다.

이 수치를 정리·회고 시간에 공유합니다.

크기가 왜 중요한가: 이미지는 배포할 때마다 네트워크로 전송됩니다.

1GB 이미지를 서버 10대에 배포하면 10GB가 오갑니다.

그리고 컨테이너가 뜨는 시간도 그만큼 길어집니다.

2 레이어를 합쳐 층수를 줄입니다

RUN을 여러 번 쓰면 층이 그만큼 늘어납니다.

특히 apt-get으로 뭔가를 설치할 때는 설치와 정리를 한 층에서 끝내야 합니다.

다음 층에서 지우면 이미 아래층에 남아 있어 크기가 줄지 않습니다.

dockerfile

```

1 # 나쁜 예 - 3개 층, 게다가 캐시 파일이 아래층에 남습니다
2 RUN apt-get update
3 RUN apt-get install -y curl
4 RUN rm -rf /var/lib/apt/lists/*
5
6 # 좋은 예 - 1개 층, 정리까지 같은 층에서 끝냅니다
7 RUN apt-get update \
8   && apt-get install -y --no-install-recommends curl \
9   && rm -rf /var/lib/apt/lists/*

```

주의 이미지 층은 지워도 사라지지 않습니다.

아래층에서 만든 파일을 위층에서 rm 하면 화면에서는 안 보이지만 이미지 크기에는 그대로 남아 있습니다.

비밀번호나 키를 실수로 넣었다가 다음 층에서 지워도 소용없는 이유가 이것입니다.

3 멀티스테이지 빌드 맛보기

빌드할 때만 필요한 도구(컴파일러 등)를 최종 이미지에 남기지 않는 방법입니다.

앞 단계에서 만든 결과물만 마지막 단계로 옮겨 담습니다.

dockerfile

```

1 # 1단계 - 빌드 전용 (무거운 도구를 마음껏 씁니다)
2 FROM python:3.12 AS builder
3 WORKDIR /app
4 COPY requirements.txt .
5 RUN pip install --no-cache-dir --target=/deps -r requirements.txt
6
7 # 2단계 - 실행 전용 (필요한 것만 가져옵니다)
8 FROM python:3.12-slim
9 WORKDIR /app
10 COPY --from=builder /deps /deps
11 ENV PYTHONPATH=/deps
12 COPY . .
13 CMD ["python", "app.py"]

```

bash

```

1 cat > Dockerfile.multi << 'EOF'
2 FROM python:3.12 AS builder
3 WORKDIR /app
4 COPY requirements.txt .
5 RUN pip install --no-cache-dir --target=/deps -r requirements.txt
6
7 FROM python:3.12-slim
8 WORKDIR /app
9 COPY --from=builder /deps /deps
10 ENV PYTHONPATH=/deps
11 COPY . .
12 CMD ["python", "app.py"]
13 EOF
14
15 docker build -t data-app:multi -f Dockerfile.multi .
16 docker run --rm data-app:multi
17 docker images data-app

```

확인 포인트: data-app:multi가 정상 실행되고, 크기가 data-app:fat보다 훨씬 작으면 성공입니다.

TIP 지금 예제는 패키지가 하나뿐이라 slim 단독 빌드와 큰 차이가 없습니다. 멀티스테이지의 효과는 컴파일이 필요한 언어(Go·Java·C++)나 빌드 도구가 무거운 경우에 큼니다. 오늘은 "이런 방법이 있다"까지만 알아 두면 충분합니다.

4 이미지 안을 들여다봅니다

bash

```

1 # 총별로 어떤 명령이 얼마의 크기를 만들었는지
2 docker history data-app:0.3
3
4 # fat 버전과 비교
5 docker history data-app:fat | head -n 8
6
7 # 이미지 안에 내 파일이 잘 들어갔는지 직접 확인
8 docker run --rm data-app:0.3 ls -la /app

```

확인 포인트: docker history에 각 층의 SIZE가 보이고, ls -la /app에 app.py·requirements.txt·Dockerfile이 나오면 성공입니다.

5 1-5 쓸 이미지를 확정합니다

bash

```
1 # 실험용 이미지 정리
2 docker rmi data-app:bad data-app:bad2 data-app:good2 data-app:fat
  data-app:multi
3
4 # 1-5 Compose에서 쓸 최종 태그를 붙입니다
5 docker tag data-app:0.3 data-app:latest
6 docker images data-app
7
8 # 마지막으로 한 번 실행해 확인
9 docker run --rm data-app:latest
```

확인 포인트: data-app 태그가 0.1·0.2·0.3·latest만 남고, docker run이 정상 출력을 내면 오늘 목표 달성입니다.

app 폴더에 Dockerfile이 있어야 1-5 Compose가 build 할 수 있습니다.

정리·회고 (0.5H)

최적화 전후 수치를 서로 공유하고, 오늘의 지시어를 한 장으로 정리합니다.

1-4 · 정리·회고 0.5H · 정리·회고 슬라이드 9장

슬라이드: <https://raw.githubusercontent.com/aebonlee/materials/main/data-eng/decks/w1d4s4.pdf>

전후 비교를 공유합니다

오늘 잔 두 가지 수치를 옆 사람과 비교해 보세요.

환경에 따라 값이 다르게 나오는데, 그 차이 자체가 이야깃거리가 됩니다.

- ① 베이스 이미지 크기 — python:3.12로 빌드한 것과 python:3.12-slim으로 빌드한 것의 SIZE 차이
- ② 재빌드 시간 — 좋은 순서와 나쁜 순서로 각각 재빌드했을 때 time의 real 값 차이
- 주차 산출물 갤러리(패들릿)에 두 수치를 올려 두면 서로 비교하기 좋습니다.

bash

```
1 # 공유할 수치를 한 번에 뽑기
2 docker images data-app --format "{{.Tag}}\t{{.Size}}"
```

Dockerfile 지시어 요약

지시어	언제 실행되나	기억할 점
FROM	맨 처음	태그를 반드시 명시한다
WORKDIR	빌드 시점	없는 경로면 만들어 준다
COPY	빌드 시점	빌드 컨텍스트 안의 파일만 가능
RUN	빌드 시점	결과가 이미지에 굳는다. 설치와 정리를 한 층에
ENV	빌드·실행 양쪽	이미지에 남으므로 비밀값은 넣지 않는다
EXPOSE	문서화만	실제 포트는 docker run -p가 연다
CMD	컨테이너 시작 시	docker run 뒤에 명령을 주면 교체된다
ENTRYPOINT	컨테이너 시작 시	교체되지 않는 고정 명령

오늘의 체크리스트

- FROM·WORKDIR·COPY·RUN·CMD 각 지시어의 역할을 설명할 수 있다.
- data-app:0.1 이미지를 빌드하고 실행해 출력을 확인했다.
- 재빌드 로그에서 CACHED 층을 찾아 캐시 동작을 확인했다.
- 일부러 나쁜 순서로 빌드해 재빌드 시간이 길어지는 것을 수치로 확인했다.
- .dockerignore를 만들어 빌드 컨텍스트를 줄였다.
- slim과 전체 베이스의 이미지 크기를 docker images로 비교했다.
- docker history로 층별 크기를 들여다봤다.
- 1-5 Compose에서 쓸 app 디렉터리(Dockerfile 포함)가 완성되어 있다.

자주 겪는 오류

주의 증상: failed to compute cache key: "/requirements.txt" not found → 원인: docker build를 실행한 위치(빌드 컨텍스트)에 COPY 대상 파일이 없습니다.
→ 해결: cd ~/data-eng/week1/app처럼 Dockerfile과 파일들이 있는 디렉터리로 이동한 뒤 docker build ... 을 다시 실행합니다(마지막 점이 컨텍스트입니다).

주의 증상: 코드 한 줄만 고쳤는데 빌드 때마다 pip install이 다시 돈다 → 원인: COPY .. 이 RUN pip install보다 앞에 있어, 소스가 바뀔 때마다 그 아래층 캐시가 전부 깨집니다.
→ 해결: COPY requirements.txt . → RUN pip install → COPY .. 순서로 바꿉니다.

주의 증상: python: can't open file '/app/app.py' → 원인: WORKDIR 설정이 빠졌거나 COPY .. 이 없어서 코드가 이미지 안에 들어가지 않았습니다.
→ 해결: Dockerfile에 WORKDIR /app과 COPY .. 이 있는지 확인하고 재빌드합니다.
docker run --rm -it data-app:0.1 ls /app으로 파일 유무를 직접 볼 수 있습니다.

예상 질문

- Q. COPY와 ADD는 뭐가 다른가요? — ADD는 압축 해제와 URL 다운로드까지 합니다. 그래서 동작이 예측하기 어려워 COPY를 기본으로 쓰라고 공식 문서가 권합니다.
- Q. 캐시를 무시하고 처음부터 빌드하려면? — docker build --no-cache를 씁니다. 의존성이 이상하게 꼬였을 때 씁니다.

- Q. 이미지에 비밀번호를 넣으면 안 되나요? — 안 됩니다. ENV로 넣든 파일로 넣든 이미지 층에 남아 docker history로 볼 수 있습니다. 비밀값은 실행할 때 -e나 시크릿으로 넘깁니다.
- Q. 태그를 안 붙이면 어떻게 되나요? — latest가 붙습니다. 1-3에서 만든 latest와 오늘 만든 latest가 다른 이미지라 헷갈리므로, 실습에서도 버전 태그를 붙이는 습관을 들이세요.

심화 과제 (선택)

- .dockerignore에 항목을 더 넣어 보고, 빌드 첫 줄의 transferring context 크기가 어떻게 달라지는지 기록해 보세요.
- app.py를 고쳐 requests로 아무 공개 API를 호출하게 만들고, 이미지를 다시 빌드해 실행해 보세요.
- CMD 대신 ENTRYPOINT를 쓴 이미지를 만들고, docker run 뒤에 명령을 붙였을 때 동작이 어떻게 달라지는지 비교해 보세요.
- 공식 문서의 빌드 모범 사례를 읽고, 오늘 만든 Dockerfile에서 더 고칠 곳을 한 가지 찾아보세요.

참고 링크

- Dockerfile 레퍼런스: <https://docs.docker.com/reference/dockerfile/>
- 빌드 모범 사례: <https://docs.docker.com/build/building/best-practices/>
- 멀티스테이지 빌드: <https://docs.docker.com/build/building/multi-stage/>

이번 주 산출물

제출은 [산출물 제출](#) 페이지에서 하고, 제출 후 [주차별 산출물 갤러리](#)의 해당 주 섹션에 [1주] 이름 — 산출물명 형식으로 올려 동료와 공유해 주세요.

Built with the LIVE method by 이애본 (Aebron Lee) · DreamIT Biz